DPOR algorithm will detect the dependency between step `(1)` by p and step `(2)` by q, and note that it seems necessary to explore sequences that start with a step of q. The DPOR algorithm will also detect the dependency between `(1)` and `(3)` and possibly note that it is necessary to explore sequences that start with a step of r.

Existing DPOR methods guarantee that the set of processes explored from the initial state is a *persistent set*. In short, a set P of processes is persistent in the initial state if in any execution from the initial state, the first step that is dependent with the first step of some process in P must be taken by some process in P. In this example, the only persistent set which contains p in the initial state is $\{p, q, r\}$. To see this, suppose that, e.g., r is not in the persistent set P, i.e., $P = \{p, q\}$. Then the execution $r.r$ contains no step from a process in P, but its second step is dependent with the first step of p, which is in P. In a similar way, one can see that also q must be in P.

In contrast, our source set-based algorithms allow $S = \{p, q\}$ as the set of processes explored from the initial state. The set S is sufficient, since any execution that starts with a step of r is equivalent to some execution that starts with the first (local) step of q. The set S is not a persistent set, but it is a *source set*. Intuitively, a set S of processes is a source set if for each execution E from the initial state there is some process *proc* in S such that the first step in E that is dependent with *proc* is taken by *proc* itself, i.e., *proc* is not preceded by any process that is dependent with *proc*. To see that $\{p, q\}$ is a source set, note that when E is $r.r$, then we can choose q as *proc*, noting that $r.r$ is not dependent with the first step of q. Any persistent set is also a source set, but, as shown by this example, the converse is not true.

Our algorithm *source-DPOR* combines source sets with sleep sets, and will explore exactly four interleavings, whereas any algorithm based on persistent sets will explore at least five (if the first explored execution starts with p), some of which will be sleep set blocked if sleep sets are used. If we extend the example to include n reading processes instead of just two, the number of sleep set blocked explorations increases significantly (see Table 4 in Section 10).

Sleep Sets. To make this section self-contained, we here briefly review the concept of *sleep sets*. Sleep sets [Godefroid 1996; Godefroid and Wolper 1991] use information about past explorations to prevent redundant future explorations. For each prefix E of the execution that is currently being explored, a sleep set is maintained. The sleep set contains a set of processes, whose exploration after E would be redundant for the reason that an equivalent execution has already been explored by the DPOR algorithm. The sleep set at each prefix E is manipulated as follows: (i) after exploring the interleavings that extend E with some process p, the process p is added to the sleep set at E, and (ii) when exploring executions that extend $E.p$, the sleep set at $E.p$ is initially obtained as the sleep set at E with all processes that are dependent with p removed. The effect is that the algorithm need never explore a step of a process in the sleep set. We illustrate this on the program of Fig. 1. After having explored executions starting with p, the process p is added to the sleep set at the empty execution, following rule (i). When initiating the exploration of executions that start with q, the process p is in the sleep set at q, according to rule (ii). Therefore, p should not be explored after q, since executions that start with $q.p$ are equivalent to executions that start with $p.q$, and such executions have already been explored.

Wakeup Trees. As mentioned, by utilizing source sets, *source-DPOR* will explore a minimal number of executions for the program of Fig. 1. There are cases, however, where *source-DPOR* encounters sleep set blocked explorations.

We illustrate this by the example in Fig. 2, a program with four processes, p, q, r, s. Two events are dependent if they access the same shared variable, i.e., `x`, `y` or `z`. Variables `m`, `n`, `l` are local. Each statement accessing a global variable has a unique label; e.g., process s has three such statements